COGNITIVE ROBOTICS PROJECT

Belief-Based Task Planning for Multi-Robot Ship Hull Cleaning

A belief-driven, closed-loop cognitive layer for defect prioritization and cleaning verification — evaluated in simulation

Team

Pritish Bhatasana
Reg. No: 23BRS1035

Siddharth Seth
Reg. No: 23BRS1038

Introduction: Hull Fouling & Structural Defects

Ship hulls accumulate biofouling (algae, barnacles), corrosion, and cracks over time. Left unmanaged, these raise fuel consumption, accelerate structural damage, and drive up maintenance cost — yet manual underwater inspection and cleaning is slow, expensive, and hazardous for divers.

Increased fuel drag

from biofouling accumulation

Corrosion & cracks

risk structural integrity

Manual inspection

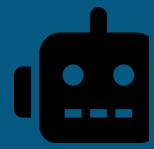
slow, costly, diver-hazardous

Our Reference System



Mapping Robot

Magnetic wheels + depth camera; scans hull, builds point cloud / mesh



Cleaning Robot

Motors + cleaning/repair kit; navigates to marked defects and acts

Inspired by prior internship work on ship hull cleaning robotics

Project Scope: Why Simulation



No original hardware / source

The mapping + cleaning robot hardware, ZED2i camera, and original RTAB-Map codebase from the internship are not available for this project.



Different ROS2 environment

Development machine runs Ubuntu 26.04 LTS with ROS2 Lyrical, not the Humble / Ubuntu 22.04 stack the original system used.



Simulation-first approach

The system is built and evaluated entirely in a Gazebo simulation — sidestepping hardware and unproven-distro risk while keeping the same problem structure.

Consequence for this project: novelty is deliberately placed in the cognition / task-planning layer — the one part of the pipeline that does not depend on unavailable hardware or an unproven ROS distro.

Literature Survey: Where the Open Gap Lies



Perception

Deep-learning defect detectors (cracks, rust, biofouling) are already well-studied — e.g. YOLOv8 hull-crack detection, LIACI segmentation benchmark.



Path Planning

Collision-aware hull navigation (enhanced A*) and SLAM-based task discovery (Map-TIDAL) are already published for this exact domain.

OUR FOCUS



Cognition

No existing work carries defect severity + confidence into task priority, or closes the loop with post-cleaning verification. This is the open gap.

Literature Survey: Positioning vs. Published Work

Paper	Domain	Key Gap vs. Our Project
Map-TIDAL (Ruediger & Banerjee, 2026)	Task discovery + allocation	No severity/confidence model; no post-action verification
Biofouling recognition + tracking (Su et al., 2024)	Perception + control	Detection fused into steering; no task-priority layer
Enhanced A* hull navigation (review, 2025)	Path planning	Collision-free routing only; no task ordering
LLM hierarchical orchestration (2026)	Cognition / orchestration	No belief model, no verification; different domain
YOLOv8 hull crack detection (2026)	Perception	Detection only, no downstream planning

Full 8-paper literature review with citations available in the accompanying document.

Problem Statement

What to clean first, in what order, and how to prove it was actually cleaned

Given hull defects of differing type, severity and detection confidence, how should a two-robot system order remediation and confirm completion — without a fixed schedule or a blind task-done assumption?

Risk blindness

Dispatch minimises travel distance only. Defect class τ(d) and severity S(d) never enter the objective, so soft biofouling at 2 m outranks a structural crack at 6 m.

Open-loop completion

Arrival at the commanded pose is treated as success. A partial cavitation pass is indistinguishable from a complete one, and the residual defect is never revisited.

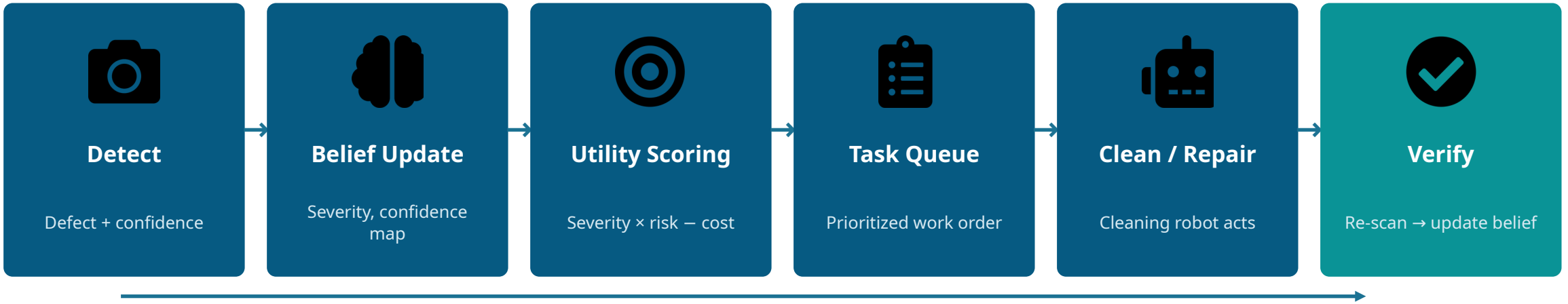
Static criticality

Severity is sampled once at detection. A crack that propagates during the mission is scheduled on stale information and can cross the critical threshold unnoticed.

Objective: maximise rank correlation r_s between the realised servicing order and ground-truth structural risk, subject to zero geofence excursions and zero inter-robot collisions — and retire no defect without physical evidence.

Proposed Work: The Self-Verifying Cognitive Loop

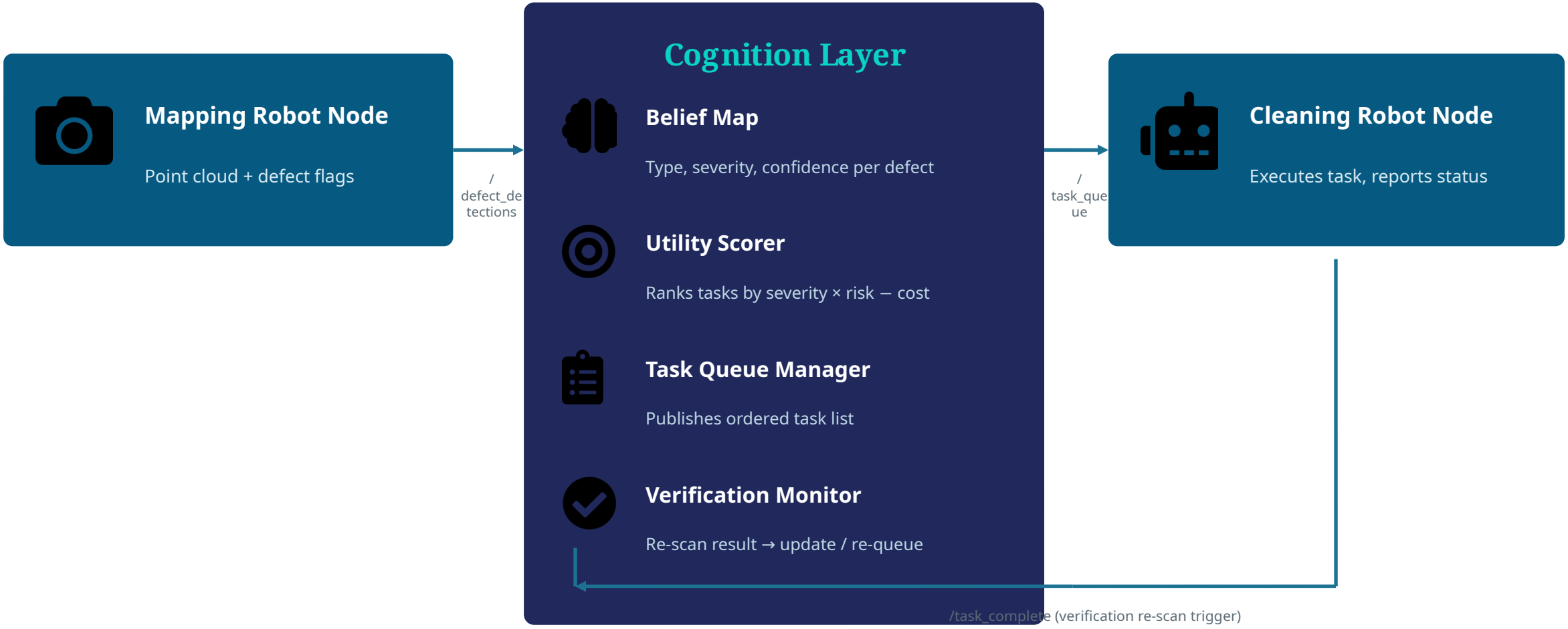
A closed cognitive loop absent from prior hull-cleaning and inspection-robotics literature



If defect still detected after cleaning → belief updated, task re-queued (loop back to Task Queue)

Why this is novel: Map-TIDAL and similar systems discover and allocate tasks by distance/keypoint density, and mark them done once visited. No reviewed paper carries severity + confidence into prioritization, or re-verifies completion — this project targets exactly that gap, in simulation.

Proposed Work: System Architecture



Two ROS2 nodes (mapper, cleaner) communicate only through the cognition layer's topics/services — a fixed specialized two-robot hand-off, not a homogeneous swarm.

Proposed Work: Utility Function & Core Algorithms

Explicit, interpretable prioritisation over true geodesic cost

$$U(d) = w_1 \cdot \hat{S}(d) \cdot W_{risk}(\tau(d)) \cdot \hat{C}(d) - \alpha \cdot g_{A^*}(p_c, p_d) + \beta \cdot \mathbb{1}_{requeued} + \Omega \cdot \mathbb{1}_{critical}$$

$\hat{S} \cdot W_{risk} \cdot \hat{C}$

Expected structural consequence: severity scaled by class risk, discounted by detection confidence.

$-\alpha \cdot g_{A^*}$

True 8-connected A* geodesic distance with 0.55 m obstacle inflation — not Euclidean distance.

$+\beta$

Escalation bonus applied when verification re-queues a defect.

$+\Omega$

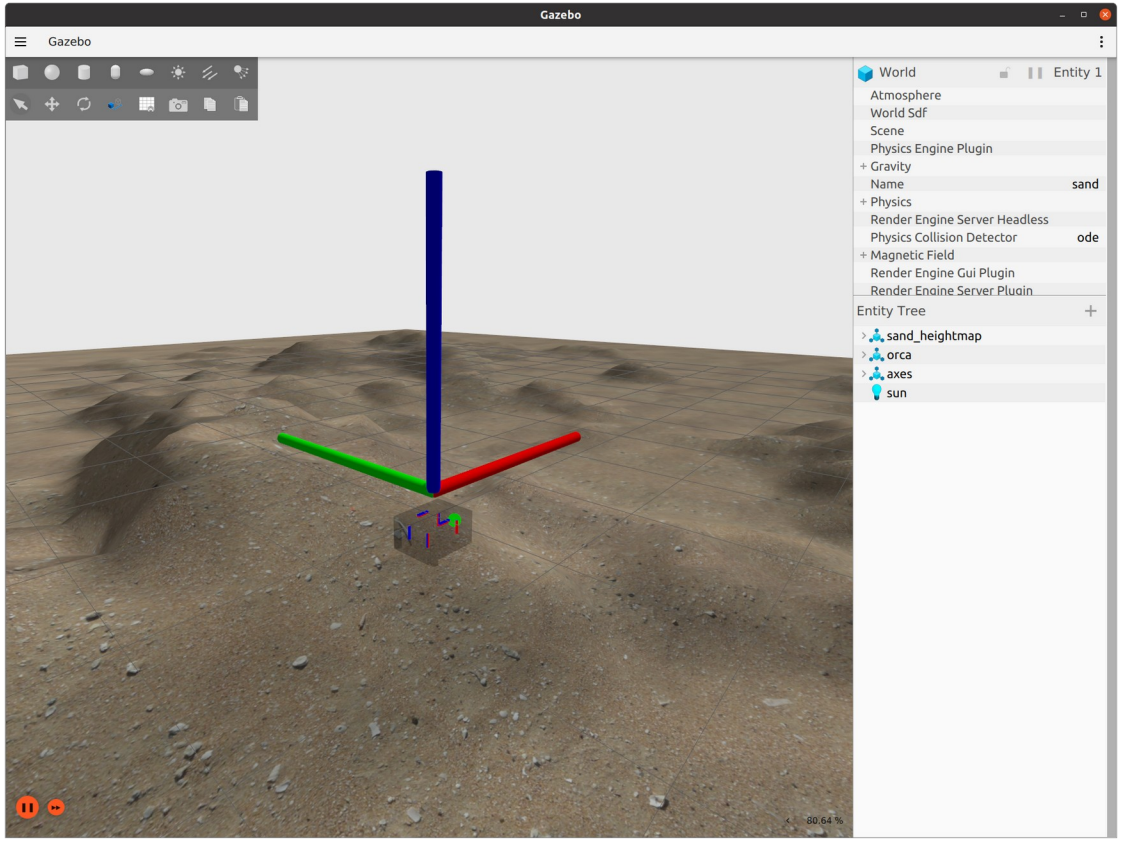
Emergency override, set an order of magnitude above the first term so it acts lexicographically.

Mechanism	Definition
Risk weight W_{risk}	crack 1.00 > corrosion 0.70 > biofouling 0.40
Belief fusion	$\hat{S}_k = (1-\lambda)\hat{S}_{k-1} + \lambda z_k$, $\lambda = 0.50$; $\hat{C}_k = \max(\hat{C}_{k-1}, c_k)$
Degradation	$dS/dt = \gamma\tau \Rightarrow$ deferred cracks genuinely propagate ($\gamma = 0.0080\ s^{-1}$)
Lane gate	$E(d) = [\ \ell(d) < \ell_{map}\ ] \vee [\ \rho(\hat{S}(d)) = \text{CRITICAL}\ ]$
Verification	residual $\leq \theta_v$ (0.08) $\Rightarrow$ VERIFIED_CLEAN, else RE-QUEUED with bonus β

Proposed Work: Simulation Environment

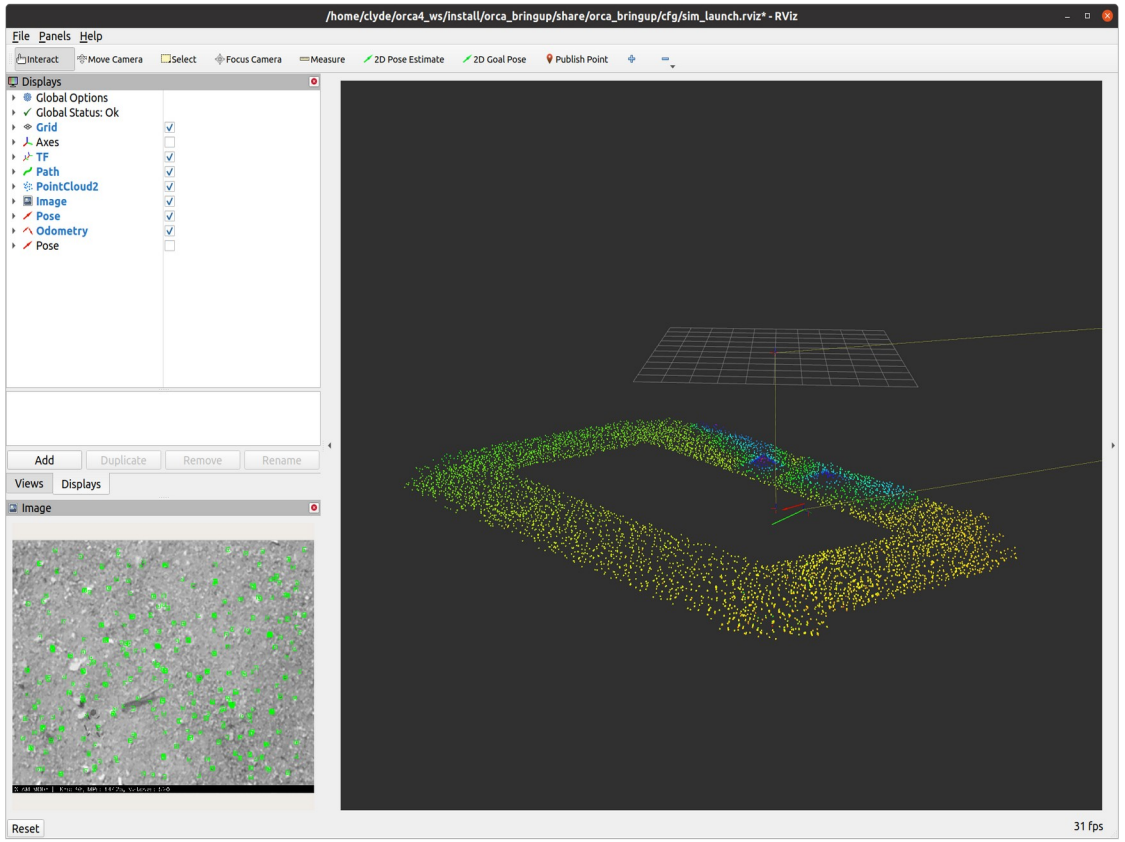
OS / ROS	Ubuntu 26.04 LTS — ROS2 Lyrical
Simulator	Gazebo (Gz Sim), synthetic hull mesh
Sensors	Simulated RGB-D camera feed (ZED2i substitute)
Defects	Synthetic texture patches: crack / rust / biofouling

Two robots (mapper + cleaner) are spawned in the same simulated world, exchanging state only through explicit ROS2 topics/services — mirroring the real two-robot handoff.



Gazebo simulation of an underwater vehicle (reference render)

Proposed Work: Mapping & Localization Layer



RViz2 visualization of robot pose, path, and sensor point cloud

Treated as a solved sub-problem

Point-to-point navigation and hull mapping are not this project's novelty target. Localization is provided either by:

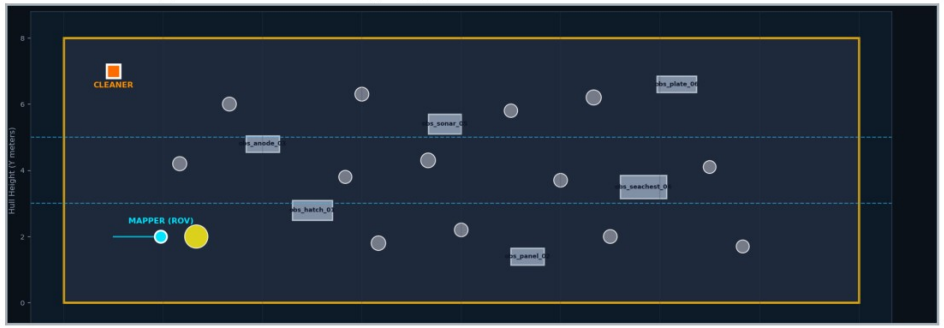
1. Gazebo ground-truth pose (fastest path to a working demo)
2. A ROS2-native SLAM stack (e.g. slam_toolbox) for a more realistic pose estimate

Either way, the mapping robot's output to the cognition layer is the same: a point cloud / mesh plus a set of candidate defect locations — the interface our belief map consumes.

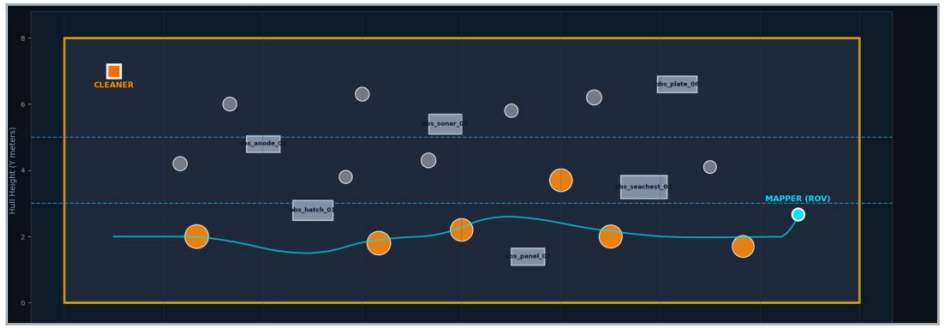
Results: Mission Walkthrough

One complete 14-defect mission — lane gating, risk ordering, preemption, verification

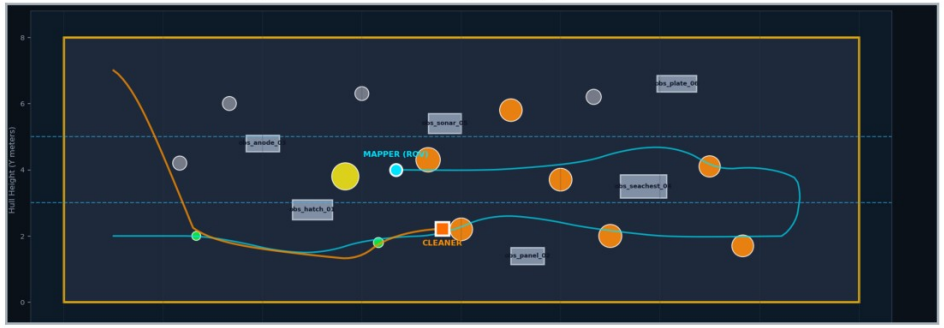
(a) $t \approx 2$ s — sweep begins, all defects UNDETECTED



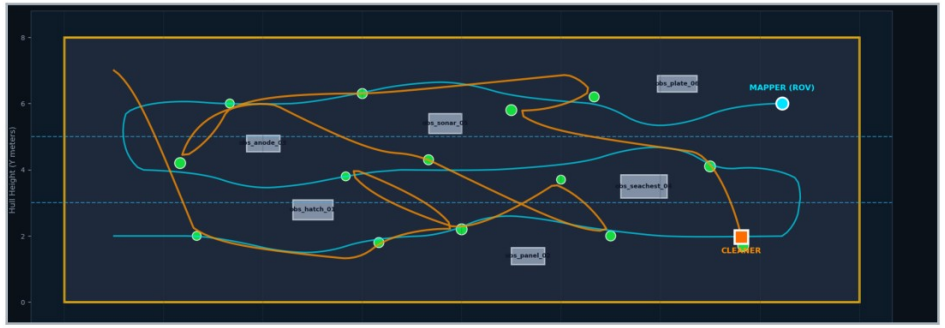
(b) Lane 1 released — cleaner authorised



(c) Escalated crack preempts active task



(d) Mission end — 14/14 VERIFIED_CLEAN



Simulator output at four mission stages. Grey = UNDETECTED, orange = QUEUED, red = CLEANING, green = VERIFIED_CLEAN. Cyan trace = mapper serpentine; orange trace = cleaner tour.

Cleaner holds until Lane 1 is released → services crack, then corrosion, then biofouling → preempts for an escalating crack via the context stack → resumes → all 14 defects verified clean.

Results: Quantitative Comparison

60 independent missions — 6 scenarios × 5 seeds × 2 architectures

Scenario	Order corr. % (Ours)	Order corr. % (FIFO)	r_s (Ours)	r_s (FIFO)	OOB / Coll.
nominal	85.3 ± 2.1	49.7 ± 1.5	+0.77	−0.07	0 / 0
swarm_scaling	60.6 ± 1.5	53.9 ± 1.4	+0.23	+0.05	0 / 0
obstacles	89.0 ± 1.5	49.7 ± 1.5	+0.83	−0.07	0 / 0
high_sensor_noise	85.3 ± 2.1	49.7 ± 1.5	+0.77	−0.07	0 / 0
agent_failure	87.6 ± 2.5	49.7 ± 1.5	+0.81	−0.07	0 / 0
marine_current	85.3 ± 2.1	49.7 ± 1.5	+0.77	−0.07	0 / 0
Overall (n = 30 each)	**82.17**	**50.38**	**+0.70**	**−0.05**	**0 / 0**

+31.8 pp

ordering correctness gain over the FIFO baseline

d = 4.37

Cohen's d; Welch t = 16.91 — a decisive effect

36%

of remediations caught and re-queued by verification

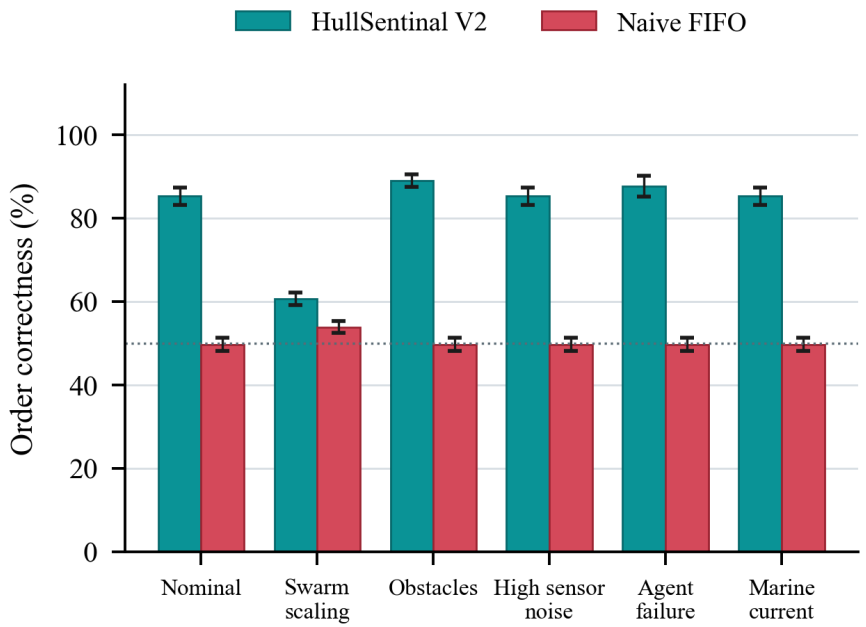
0

geofence, lane and collision violations in 60 missions

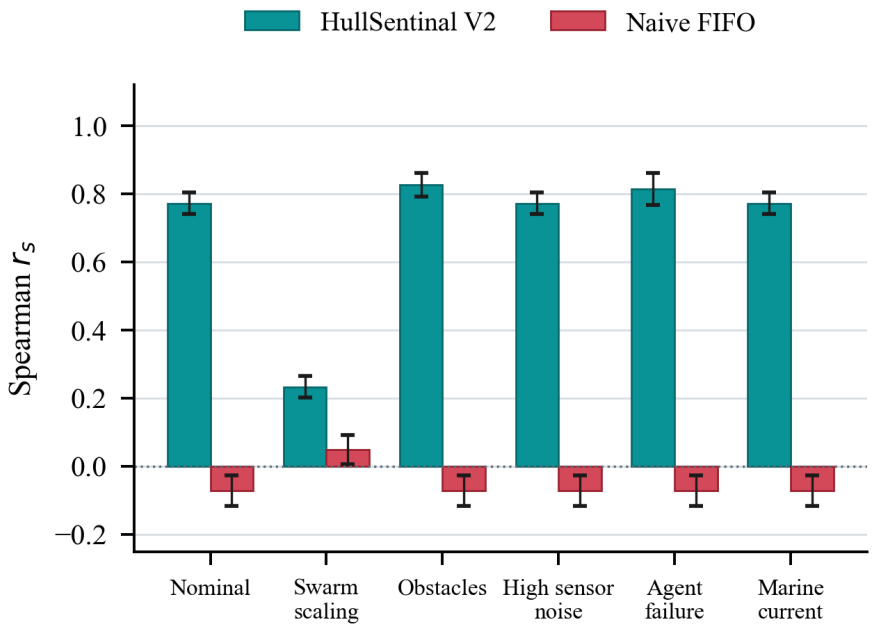
Both architectures retire all 14 defects in every mission; the ordering gain is not bought by sacrificing completeness.

Results: Ordering Quality & Risk Alignment

The baseline is pinned near chance in every scenario; the proposed system is not



Defect-ordering correctness by scenario (mean ± SD, 5 seeds).



Spearman r_s between servicing order and structural risk.

The single weak case is swarm_scaling (60.6%): three cleaners draw concurrently from one ranked queue, so completion order is partly governed by travel time. The remedy is architectural — replace the shared queue with a consensus-based bundle auction (CBBA) in which each cleaner bids on its own marginal geodesic cost.

Discussion: Trade-offs & Limitations

What the prioritisation costs, and where the claims stop

The cost of prioritisation

Missions take 106.7 s against 76.4 s for FIFO — a 39.7% increase. Lane gating idles the cleaner for separation, risk ordering abandons distance optimality, and verification adds a re-scan per defect. Each buys a property the baseline lacks, and the coefficient α moves the operating point directly.

Why the trade is favourable

In the agent_failure scenario the open-loop baseline reports 100% success while leaving residual defects on the hull. Closed-loop verification rejected 36% of remediations there. For a task where a deferred crack is the dominant risk, 30 s is a cheap price for correct ordering and verified completion.

Simulation only

Perception is a noisy oracle, not a learned detector; noise robustness characterises the belief filter, not a vision pipeline.

Planar manifold

Real hulls are doubly curved. Geodesic planning over a triangulated mesh is required before path costs transfer.

Single-server metric

Order correctness assumes one sequential cleaner and therefore under-reports multi-cleaner performance.

Utility weights were set by design reasoning rather than fitted to data, and are presented as an interpretable default rather than an optimum.

Achieved Outcomes & Impact



Novel contribution

A belief-driven, severity-aware prioritization layer with closed-loop verification, measured at 82.17% ordering correctness against 50.38% for an unprioritized baseline.



Fully reproducible in simulation

The full 14-node ROS 2 stack runs headless or with RViz2; 60 benchmark missions reproduce from a single command with fixed seeds.



Clear comparison baseline

Benchmarked against a naive FIFO baseline sharing identical physics, defects and controllers, isolating the cognition layer: Spearman r_s +0.70 vs −0.05.



Extensible design

Belief map and utility scoring remain detector-agnostic — a learned segmentation model can replace the oracle detector without redesigning the cognition layer.

Conclusion & Future Work

HullSentinal replaces geometric dispatch and self-reported completion with an explicit cognition layer: a probabilistic defect belief carrying severity and confidence, an interpretable structural-utility ranking over true geodesic cost, and a mandatory physical re-scan before any defect may be retired.

82.17%

ordering correctness, from 50.38% (FIFO)

$r_s = +0.70$

risk alignment, from −0.05 (FIFO)

14 / 14

defects verified clean every mission

0

safety violations across 60 missions

Future work

Swarm scaling

Replace the shared ranked queue with a consensus-based bundle auction so N cleaners bid on bundles by marginal geodesic cost.

3D curved hulls

Replace the planar occupancy grid with geodesic planning over a triangulated hull mesh and surface-normal adhesion.

Real perception

Substitute the oracle detector with a learned segmentation model; the cognition layer is detector-agnostic and needs no redesign.

References

- **[1]** H. Su, S. Liu, L. Zhang, Y. Chen, and C. Yang, “Biofouling recognition and boundary tracking control for underwater cleaning robots,” Ocean Engineering, vol. 295, art. 116707, 2024.
- **[2]** “Automated detection and classification of cracks in ship hull structures using deep learning,” Ocean Engineering, 2026.
- **[3]** “Detection of cracks and corrosion for automated vessel visual inspection,” LIACI dataset, EU FP7 MINOAS project.
- **[4]** “Design, perception, planning and control for underwater cleaning robotics: progress, challenges and trends,” Ocean Engineering review, 2025.
- **[5]** M. Chwełatiuk et al., “Algorithm for determining the position of a ship hull-cleaning robot,” Applied Sciences, vol. 15, no. 10, art. 5705, 2025.
- **[6]** B. P. Gerkey and M. J. Matarić, “A formal analysis and taxonomy of task allocation in multi-robot systems,” Int. J. Robotics Research, vol. 23, no. 9, pp. 939–954, 2004.
- **[7]** H.-L. Choi, L. Brunet, and J. P. How, “Consensus-based decentralized auctions for robust task allocation,” IEEE Trans. Robotics, vol. 25, no. 4, pp. 912–926, 2009.
- **[8]** M. Ruediger and A. G. Banerjee, “Mapping-guided task discovery and allocation for cooperative robotic inspection of underwater structures (Map-TIDAL),” arXiv:2602.02389, 2026.
- **[9]** “A centralized multi-robot task allocation for industrial plant inspection by using A* and genetic algorithms.”
- **[10]** E. Galceran and M. Carreras, “A survey on coverage path planning for robotics,” Robotics and Autonomous Systems, vol. 61, no. 12, pp. 1258–1276, 2013.
- **[11]** “A hierarchical LLM-based framework for heterogeneous multi-robot orchestration in high-risk energy facility maintenance,” 2026.
- **[12]** P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” IEEE Trans. Systems Science and Cybernetics, vol. 4, no. 2, 1968.
- **[13]** S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot Operating System 2: design, architecture, and uses in the wild,” Science Robotics, vol. 7, no. 66, 2022.
- **[14]** N. Koenig and A. Howard, “Design and use paradigms for Gazebo, an open-source multi-robot simulator,” in Proc. IEEE/RSJ IROS, 2004, pp. 2149–2154.

Entries [1]–[5], [8], [9] and [11] are the project’s domain sources; [6], [7], [10], [12]–[14] are the methodological references cited in the paper.

Thank You

Questions & Discussion

Pritish Bhatasana
23BRS1035

Siddharth Seth
23BRS1038